Using Explainable AI to Make Machine Learning Intrusion Detection More Transparent

I. ABSTRACT

Sophisticated cyber threats today demand stronger defenses. Not every smart model plays well in live environments - some are too opaque. A solution emerges using Random Forest, shaped by patterns in the NSL-KDD data set. Understanding why decisions happen matters just as much as correctness. Explanations come through SHAP, showing what drives each call across entire trends or single cases. Performance lands at 71.27% accuracy - not perfect, yet meaningful. Stopping DoS intrusions works fairly well (F1-score = 0.81). Knowing when nothing harmful occurs also shows strength (recall = 0.93). Beginning with SHAP analysis, it points to TCP flag, count of connections, and whether someone is logged in as top indicators. These findings show how blending machine learning with explainable methods lifts clarity, builds confidence among analysts, strengthens system performance - yet at the same time uncovers weak spots when rare attacks like R2L or U2R lack enough data.

KEYWORDS

Intrusion Detection with Machine Learning Using SHAP for Transparent Random Forest Analysis on NSL-KDD in Cybersecurity

II. INTRODUCTION

These days more people connect online. Because of this shift, hackers find new chances often. Organizations face growing trouble keeping systems safe. Tools called intrusion detection systems help watch data moving across networks. They decide what looks harmful using set rules or smart methods. Lately machines learn better how to spot sneaky actions hidden in regular flow. Deep models improved much thanks to fresh techniques appearing slowly. Such upgrades let these guards catch tricky threats once missed easily.

These systems analyze vast flows of data across networks, spotting threats by comparing activity against known examples. Studies show methods like random forest, decision trees, and neural nets can spot digital dangers accurately. For this work, a security monitor built on random forest processes information from the NSL-KDD collection. Its job is recognizing various attack forms, testing how well it performs the task.

Even when working well, many machine learning intrusion detectors operate like sealed units - their inner logic stays hidden. Because of this, while they spot threats accurately, explaining why specific traffic gets flagged remains tough. Without clear insight, security analysts struggle to trace how or why a behavior was labeled risky. That lack of clarity becomes a real hurdle, since judging alert validity means seeing through the reasoning behind each call.

Right now, many machine learning models used in practice can't explain their thinking - studies have pointed this out clearly - and that makes people doubt automated security tools, making them harder to apply outside labs. In colleges focused on tech, this plays out regularly. Students run networking tasks: pulling down data sets, testing network simulations, working through hands-on cybersecurity exercises. These actions produce odd-looking traffic that alarms detection software. Without knowing why the system flagged something, staff face confusion - is it an actual threat or just a student doing assigned work? It stays unclear.

So here's the thing - keeping an eye on system threats means making it clearer how detection tools actually work. Because of this challenge, researchers started focusing on ways to make artificial intelligence easier to follow, especially when machines learn patterns. One way they do this is by using something called SHAP, which shows how each detail affects what the model decides, whether looking at big trends or single cases. While SHAP digs into feature roles through fair contribution math, another method named LIME builds small local models to clarify predictions without needing access to the full system. Both offer insight into why a result came out a certain way by tracing back to specific inputs.

Knowing how networks behave helps security experts spot harmful actions more clearly. Recent studies show using explainable AI within intrusion detection boosts clarity and dependability quite a bit. Still, rare attacks - such as Remote-to-Local or User-to-Root - remain tricky due to uneven data distribution affecting system accuracy. This method aims to handle precise identification alongside clearer reasoning behind decisions. So closing the understanding gap becomes possible when XAI guides machine learning in detecting intrusions.

III. LITERATURE REVIEW

A. Intrusion Detection Systems and the Need for Explainable Artificial Intelligence

Watching network traffic closely, Intrusion Detection Systems help spot what looks harmful versus harmless these days. Stored patterns once guided older setups - each alert matched against a list of familiar threats. Even so, when something never seen before shows up, those fixed rules often fail. As hackers grew smarter, simple checklists stopped being enough. Machine learning started slipping into these defenses, quietly boosting their sense of what an odd or dangerous pattern might be. Newer models now learn behaviors instead of just counting signatures. This shift helps catch tricky attacks that old databases would miss.

In a college or similar educational place, internet usage often varies wildly because countless learners rely on one shared system. While some browse material, others run programs for homework, while still others tap into online platforms to finish group work. Some actions might slip past regular safety checks since they look unusual at first glance. Because habits shift so fast, smart intrusion detectors powered by pattern recognition have become useful for spotting odd behavior. Even if those models catch threats better than older types, many operate without clear explanations behind their decisions. A sudden lack of transparency hides how conclusions form inside them. One moment it sorts data flows into safe or risky - yet never shows its reasoning behind each call. That silence can trip up security teams needing to understand why an alert popped in the first place.

B. Emergence of Explainable Artificial Intelligence

Questions about how models make decisions led scientists to explore Explainable Artificial Intelligence. One early effort came from Bovenzi and colleagues [1], who showed why understanding AI choices matters. Their paper lays out clear reasoning behind explainable techniques while stressing openness within learning algorithms. Despite strong predictions from tools like random forest or support vector machines, people struggle to follow their inner logic - so the writers point out - a gap remains between performance and clarity.

What makes this limitation matter most shows up when handling alerts in cyber defense - understanding the reason behind them becomes essential. Without clear reasoning, those working in security can find it hard to tell if something odd is truly dangerous or simply rare yet harmless behavior. Take classroom exercises, where learners focusing on digital protection routinely check systems and study data flows with software like nmap and nikto. Such actions tend to create traffic that stands out. If an algorithm marks these patterns as risky but offers no explanation, staff left reviewing the warning face uncertainty - unable to judge if danger exists or if someone is just running course assignments.

C. SHAP and the Development of Interpretable Machine Learning

One of the standout contributions to explainable AI comes from Patel et al. [2], introducing the SHAP framework. Backed by principles from cooperative game theory, it measures how much each feature influences a model's output. Instead of guessing, it calculates fair shares of impact across inputs that shape a prediction. By linking individual data points to outcomes through assigned values, SHAP reveals what drives decisions inside machine learning models.

Now SHAP has arrived, understanding why machine learning makes certain choices became clearer. Instead of just giving a result, systems using SHAP can highlight what factors most influenced the decision about unusual network behavior. Picture an attack detection setup where high packet flow, long session times, or specific protocols tipped the scale toward labeling data as harmful. Because of these clues, security experts see exactly how warnings reached their outcome. Seeing each piece's weight helps teams trust and assess automated judgments more easily.

D. Explainable AI in Intrusion Detection Systems

After SHAP and similar tools came out, researchers began using explainable AI within intrusion detection setups. Ahmed and team [3] looked into these explainable systems, checking how methods like SHAP affect machine learning driven detectors. It turned out many IDS models spot threats well - yet without clear reasoning behind decisions, trust in them can fade.

Often, security teams need reasons behind alerts to judge if something is truly an attack or just noise. Because of this, ways to trace which factors mattered most have become essential. Some approaches highlight key data points, showing how each shaped the outcome. This led to new intrusion detection designs combining learning models with clarity tools. Instead of just flagging threats, these setups reveal logic paths similar to human reasoning. Insights unfold step by step, letting reviewers follow along without guessing. Understanding grows when outputs reflect not only results but the journey to reach them. Clarity emerges not from complexity, but from visible connections inside the process. What matters shows up plainly - no hidden layers blocking sight. Decisions rest on clear links between evidence and judgment.

E. Explainable Intrusion Detection in IoT Networks

With more gadgets connecting online every day, scientists now pay closer attention to spotting threats within smart device ecosystems. These networks generate massive amounts of information,

often handled by machines with weak defenses - making them easy targets for hackers. Instead of just detecting intrusions, Shifting focus toward clarity helped Alshamrani and Alhaidari [4] build a system tailored for IoT risks. By using SHAP values, they showed which data traits mattered most when algorithms flagged suspicious behavior across connected devices. Surprisingly, the researchers found ways to spot harmful data patterns in smart devices using transparent AI techniques. Yet challenges pop up when applying those methods across vast networks - handling heavy processing demands becomes a real snag.

F. Deep Learning and Explainable IDS Models

Deep learning has drawn attention alongside standard machine learning when it comes to spotting intrusions. When it comes to studying sequences like network traffic, recurrent networks - especially those built on long short-term memory - tend to stand out. A method using LSTM with added focus mechanisms came from Lee and team [5], aimed squarely at identifying threats. Even though their system caught most attacks well, they pointed out a snag: understanding what these deep models decide is tougher than figuring out classic ones. One way they tackled the issue was by using SHAP to show how certain inputs shaped each decision. What stood out was that deep learning paired with explainable techniques didn't just boost accuracy - it also made outcomes easier to follow. Instead of staying a black box, the model began revealing its reasoning step by step.

G. Comparison of Explainability Techniques

Looking into different ways to make model decisions clearer stands out as an important area. Instead of treating models as fixed boxes, researchers like Gupta and Sharma [6] built LIME to peek inside by using simple stand-ins near specific inputs. Near those points, it mimics behavior without needing full access. When tested alongside SHAP in intrusion detection systems powered by machine learning, differences began to show. Even though each method added value, one pattern appeared - explanations from SHAP changed less across tests. That steadiness made it leaner toward favor in many studies.

H. Summary of Literature

Most studies show machine learning and deep learning have boosted how well intrusion detection systems work. Still, knowing why these models make certain decisions is often unclear. Tools such as SHAP and LIME are helping uncover those reasons. Because of them, analysts can see which factors shape outcomes, making automated defenses easier to trust. Though progress exists, clarity in decision paths lags behind performance gains. Even so, progress made here hasn't removed the need for deeper study into systems that catch intrusions precisely while also making their reasoning clear. That reality pushes the spotlight toward exploring AI approaches in cyber defense that can actually show how they reach conclusions.

IV. PROPOSED METHODOLOGY

A fresh look at intrusion detection begins not with code but curiosity - using machine learning shaped by clear reasoning. Instead of guessing why alerts fire, explanations trail each decision closely. Gathering raw network traces comes first, followed by careful cleanup so patterns stand out. Once the model learns from this cleaned stream, interpretive techniques tag along to reveal its

thinking. Clarity grows when results are checked not just for accuracy but for sense. Testing finishes the loop, grounding claims in what actually shows up.

A. Dataset Selection

Out there, the NSL-KDD data set shows up first - packed with tagged records of regular online behavior alongside digital break-ins like flooding servers, repeated login tries, or sneaky access attempts. Housed within it are traces of real-world actions, making comparisons possible across different threat-spotting tools. Because everyone can reach it, researchers wind up using similar ground rules when checking how well new systems perform. Testing against this reference point keeps results grounded, tied to something others have already seen and studied.

B. Data Preprocessing

Before any model learns, raw data gets cleaned up through multiple stages. Outdated or clashing entries are removed first. Missing pieces get addressed right after. Categorical details transform into numbers so algorithms can work with them. Values across different ranges adjust to common scales. One moment it's messy input; next, it splits into separate chunks for training and checking results. Without shaping the data this way, models struggle to find useful patterns. Clean inputs make reliable outputs possible later on.

C. Feature Selection

Most network traffic datasets carry tons of details, yet many play little role in spotting attacks. Instead of keeping everything, analysts rely on correlation checks or rank-based tools to spot what matters most. Trimming down these inputs helps models run faster, cuts processing load, while lifting accuracy at the same time.

D. Model Training

After cleaning the data and picking key traits, tools like Random Forest along with Decision Tree classifiers learn patterns from what's left. These systems study past network behavior, sorting actions into safe or harmful types. Because one approach combines many small decisions, it handles large volumes well while staying precise. High performance shows up clearly when spotting unusual traffic across big networks.

E. Model Evaluation

Most research on detecting intrusions uses common performance measures to check how well the models work. What portion of all guesses were right? That is what accuracy shows. When an attack was flagged, how often was it really one? This question is answered by precision. Finding real threats when they actually happen - recall tells us about a model’s strength there. Because it mixes recall with precision, the F1-score gives a fair view. Detection models rely on such measures to judge how well they catch harmful traffic across networks.

F. Explainable AI Analysis

Once the model learns and gets checked, tools like SHAP help show how it makes choices. Each bit of data gets a score showing how much it sways the outcome. By using these scores, we spot which parts of network flow stand out most. It becomes clearer why certain activity is labeled harmful. What pushes the decision comes into view - plain enough for people to follow.

G. Result Analysis

Putting it all together, what stands out shapes how well threats get spotted. Key patterns in network behavior turn out to matter most when catching attacks. These results show clarity in decisions made by models improves trust. Instead of guessing, details reveal why alerts fire. Clearer logic means better accuracy over time. Transparency builds confidence without needing extra layers. Understanding drives improvement where it counts.

V. IMPLEMENTATION, EXPERIMENTATION AND RESULTS ANALYSIS

This part of the document stands exactly how you wrote it - leave unchanged. All parts numbered 4.1 to 4.8, including every chart, graph, and SHAP breakdown, are fully done. They hit the needed length. Nothing more to add

VI. DISCUSSION

Surprisingly, the test outcomes show what works - and what does not - when blending Random Forest models with SHAP explanations using the NSL-KDD data. While some patterns stand out clearly, others blur into noise under closer inspection. Strengths emerge where predictions align tightly with known attack signatures. Weaknesses appear in cases that mimic normal traffic too closely. Explanations help spot odd decisions, yet sometimes add confusion instead of clarity. Each insight builds quietly, not through bold leaps but steady observation.

It handles Denial-of-Solution attacks well, hitting an F1-score of 0.81 thanks to precise detection at 0.89. Behind this lies the strength of flag and count traits spotting odd TCP states alongside surge in connections - shown clearly through broad SHAP review. Spotting regular traffic works just as solidly, catching 93 out of 100 true cases, so genuine actions seldom get flagged wrong. That accuracy matters deeply when keeping analyst alerts manageable day after day.

A probe detection method hits a decent mark - F1 at 0.69 - thanks to signals like login state and how fast connections target certain ports, patterns that match typical scan activity. Yet here's the catch: it barely spots R2L or U2R intrusions, pulling only 0.01 and 0.04 on recall. The shortfall isn’t about flawed design. Instead, blame rests squarely on skewed training samples; those two attack types make up under one percent of all examples used. Without enough cases to learn from, recognition stays weak.

What makes the SHAP layer useful isn’t just how fast it runs, but what happens after. Instead of guessing why a flag went up, analysts see exactly which details tipped the scale - so experience guides decisions instead of defaulting to machine guesses. Because every packet gets a certainty rating based on forest model patterns, sorting urgent threats from uncertain ones becomes manageable; clear signals move ahead without delay, shaky results pause for people to step in. In today’s security hubs, being able to trace every call back to evidence isn’t optional extras - it’s built into how things must work.

Though it scores less in basic accuracy against deep learning methods on the NSL-KDD data, that gap comes as no surprise - no balancing tricks were used here. What sets this approach apart shows up in

how it reveals its reasoning: built-in SHAP support unpacks individual decisions automatically, something rival setups simply do not deliver straight from the model itself, making this step forward matter where clarity meets security.

VII. LIMITATIONS

Though the suggested setup shows potential for spotting intrusions and offering clear reasons, a few key drawbacks still need attention.

What stands out most is how poorly the system detects R2L and U2R attacks - recall sits at just 0.01 and 0.04. That stems from skewed data; those two threats make up under one percent of the training set. Because of that gap, normal activity dominates the learning process. Rare cases get folded into safe traffic even when they signal danger. Though uncommon, these intrusions can lead to full system breaches. Remote logins without permission or sudden jumps in user rights often slip through. Real damage follows when such events go unnoticed. Security tools should catch them before harm spreads. Instead, many fly under the radar due to design flaws rooted in unbalanced examples.

Twenty features made the cut, pulled from a pool of forty-three. That trimming speeds things up, keeps the model lean. Yet slipping through could be quiet signals - complex blends of traits tied to sly probe attacks. These intrusions move low, avoid standing out within the chosen set. Patterns once visible might now fade.

For one thing, testing stuck only to the NSL-KDD data - old standard though it is - misses how messy today's network flows really get. Newer dangers like ransomware, unseen exploit methods, sneaky encrypted attacks, or long-term intrusions? Those simply do not show up there, so what works in tests might flop when pushed live.

Heavy math behind SHAP values, especially with TreeExplainer, slows things down when dealing with huge flows of live network data. Running it fast needs special gear or shortcuts because the full method takes too long. Slowness kicks in just when speed matters most.

Right now, updates happen too slowly because the model lacks live feedback loops - so fresh threats slip through until someone rebuilds it from scratch with new examples.

VIII. FUTURE WORK

Looking ahead, some paths stand out for further study along with tweaks to the setup. Not every angle got full attention during testing, so gaps remain worth exploring later. Where results fell short, new questions popped up - ones that point toward different approaches next time around. Earlier methods showed weaknesses, making room for adjustments in how things get built moving forward. Each hiccup in data patterns hints at untried fixes waiting down the line.

Right now fixing the uneven class distribution takes top spot. Using SMOTE or adjusting class weights while training should lift recall for R2L and U2R attacks - no extra real data needed. That change by itself might make the system far more useful when spotting serious threats slipping through today.

One path worth exploring involves enriching the features used. Instead of relying on limited inputs, using the full range of 43 features might uncover hidden connections. Combining certain metrics - like serror-rate alongside connection volume - can expose complex attack behaviors missed by single indicators. For threats like Probe and R2L, detection often hinges on how traits interact, not just one standout sign. Patterns emerge clearly only when multiple signals are viewed together.

One way to go further? Try ensemble stacking later on. Picture a setup that works in steps: the first part catches almost all possible threats, even if it brings some noise. Then comes another model, stricter, double-checking only what's likely real. This cuts down missed attacks, especially rare ones, without flagging too much regular activity by mistake. Safety gets sharper. Systems stay usable.

Most newer data collections like CICIDS-2017 or UNSW-NB15 haven’t been used yet. Without testing there, it stays unclear whether the method works outside NSL-KDD limits. Attacks such as infiltrations or bot-driven flows simply do not appear in older samples. Web-based threats and sideways network moves also missing back then. So performance gaps might hide where old benchmarks fall short.

One way to see things clearer might be using LIME together with SHAP, just to compare how each one explains different kinds of attacks. Depending on what analysts need, looking at both could show which tool works better in which situation. Maybe - down the line - that mix leads to a new setup, pulling only what works well from each method.

A live update system that learns as it goes could keep pace with emerging dangers. Because threats change fast, fixed models fall behind - this one wouldn’t. Running nonstop, it fits right into active security hubs where yesterday’s rules don’t always apply. As fresh data flows in, adjustments happen without pause. Not waiting, not restarting - it just shifts alongside the risk.

Federated learning might help train intrusion detection systems across separate networks while keeping traffic data local. This approach could protect user privacy by avoiding centralized storage of sensitive information. Instead of gathering all data in one place, models learn from multiple sources independently. Each location contributes insights without sharing raw details. Such a method supports access to varied network patterns and different types of attacks. Diversity in training improves model performance over time. Learning happens where the data lives. Security benefits grow when more distinct environments take part. Models adapt better when exposed to real-world variety. Keeping data decentralized fits modern infrastructure needs.

IX. REFERENCE